🧠 Coordinate Systems in MLX Python

Why This Topic Is Important

One of the biggest difficulties when starting graphical projects with MLX Python is understanding the difference between:

- the logical coordinates used by the game or maze
- the real pixel coordinates used by the window

Most bugs in tile-based games come from mixing these two systems incorrectly.

Understanding coordinate systems correctly makes it much easier to:

- draw maps
- move the player
- animate entities
- center the maze
- detect collisions
- scale the game window
- render assets correctly

📦 The Two Main Coordinate Systems

In MLX projects, you normally work with:

1️⃣ Grid Coordinates (Logical Coordinates)

These coordinates represent positions inside the maze or game logic.

They are NOT pixels.

They simply describe where something exists in the map structure.

Example maze:

```
# # # # #
# P . . #
# . # . #
# . . E #
# # # # #
```

If the player is here:

```
P
```

its logical position could be:

```
player_x = 1
player_y = 1
```

because the player is located on tile `(1, 1)` inside the grid.

2️⃣ Screen Coordinates (Pixel Coordinates)

These coordinates are the REAL positions inside the MLX window.

MLX does not understand maze tiles.

MLX only understands pixels.

For example:

```
mlx.put_image(img, x, y)
```

The values `x` and `y` are pixel positions.

Example:

```
mlx.put_image(player, 128, 64)
```

means:

- draw the player 128 pixels from the left
- draw the player 64 pixels from the top

🔄 Converting Grid Coordinates to Pixels

This is the most important formula in tile-based games.

If each tile has a size of:

```
TILE_SIZE = 64
```

then every logical coordinate must be converted into pixels.

Formula:

```
screen_x = grid_x * TILE_SIZE
screen_y = grid_y * TILE_SIZE
```

📐 Example

Player logical position:

```
grid_x = 3
grid_y = 2
```

Tile size:

```
TILE_SIZE = 64
```

Calculation:

```
screen_x = 3 * 64
screen_y = 2 * 64
```

Result:

```
screen_x = 192
screen_y = 128
```

Final rendering:

```
mlx.put_image(player_img, 192, 128)
```

🎮 Why Tile Size Exists

Tile size defines how large each maze block appears on the screen.

Example:

```
TILE_SIZE = 32
```

means:

- every wall image is 32x32
- every floor image is 32x32
- movement happens in blocks of 32 pixels

Larger tile sizes:

✅ easier to see ✅ larger sprites ❌ fewer tiles fit on screen

Smaller tile sizes:

✅ larger maps ✅ more visible area ❌ harder to see details

🧱 Movement by Tile

Most MLX maze projects move using logical coordinates.

Example:

```
player_x += 1
```

This does NOT move the player by 1 pixel.

It moves the player by:

```
1 TILE
```

The pixel conversion happens only during rendering.

This is extremely important.

The game logic should normally work in grid coordinates.

The renderer converts everything into pixels.

❌ Common Beginner Mistake

Many beginners accidentally mix logical coordinates with pixel coordinates.

Example WRONG approach:

```
player_x += 64
```

This creates problems because now:

- collision systems break
- maze indexing breaks
- pathfinding breaks
- coordinates become inconsistent

Correct approach:

```
player_x += 1
```

Then later:

```
screen_x = player_x * TILE_SIZE
```

🧩 Centering the Maze on Screen

Another important concept is centering.

If your maze is:

```
maze_width = 20
maze_height = 10
```

and each tile is:

```
TILE_SIZE = 64
```

then the maze pixel size becomes:

```
maze_pixel_width = maze_width * TILE_SIZE
maze_pixel_height = maze_height * TILE_SIZE
```

Example:

```
maze_pixel_width = 20 * 64 = 1280
maze_pixel_height = 10 * 64 = 640
```

If the window is:

```
WINDOW_WIDTH = 1600
WINDOW_HEIGHT = 900
```

you can center the maze:

```
offset_x = (WINDOW_WIDTH - maze_pixel_width) // 2
offset_y = (WINDOW_HEIGHT - maze_pixel_height) // 2
```

This creates margins around the maze.

🎨 Rendering With Offsets

When centering exists, rendering changes slightly.

Instead of:

```
screen_x = grid_x * TILE_SIZE
```

it becomes:

```
screen_x = offset_x + (grid_x * TILE_SIZE)
screen_y = offset_y + (grid_y * TILE_SIZE)
```

This moves the entire maze to the center of the screen.

⚠️ Coordinate Origin (0,0)

In MLX:

```
(0,0)
```

is located at:

```
TOP LEFT CORNER
```

Important:

- X increases to the RIGHT
- Y increases DOWNWARD

This is different from traditional mathematical graphs.

Visual representation:

```
(0,0) ──────────► X
  │
  │
  │
  ▼
  Y
```

🐍 Recommended Architecture

A good MLX Python project usually separates:

Game Logic

Handles:

- maze structure
- player coordinates
- collisions
- BFS
- pathfinding
- movement

Works mostly with:

```
(grid_x, grid_y)
```

Renderer

Handles:

- images
- drawing
- animation
- UI
- scaling

Works mostly with:

```
(screen_x, screen_y)
```

🔥 Real Problems Caused by Bad Coordinate Systems

Common bugs:

- player appears between tiles
- walls overlap
- path animation misaligned
- banner stretching
- collision not matching visuals
- player movement clipping walls
- maze not centered
- seams between textures

Most of these happen because logical coordinates and pixel coordinates become mixed together.

✅ Final Recommendation

Always think like this:

```
Game Logic → Grid Coordinates
Rendering → Pixel Coordinates
```

This separation makes the project:

- cleaner
- easier to debug
- easier to scale
- easier to animate
- easier to maintain

It is one of the most important concepts in MLX Python development.